

Messages > Tools

Define tools

Specify tool schemas, write effective descriptions, and control when Claude calls your tools.

 Copy page 

Prerequisites

- Familiarity with the [tool use overview](#)
- A Claude API key and a working SDK or cURL setup

Choosing a model

Use the latest Claude Opus model, Claude Opus 5, for complex tools and ambiguous queries; it handles multiple tools better and seeks clarification when needed.

Use Claude Haiku models for straightforward tools, but note they may infer missing parameters.

 If using Claude with tool use and thinking, see [Thinking](#) for more information.

Specifying client tools

Client tools are specified in the `tools` top-level parameter of the API request. Anthropic-schema client tools, such as the bash and text editor tools, are declared by a date-versioned `type`; see each tool's page, linked from the [Tool reference](#), for the fields it accepts. The computer use and browser use tools are [client toolsets](#): a single entry with no `name` that declares a fixed set of member tools. A user-defined tool definition includes:

Parameter	Description
<code>name</code>	The name of the tool. Must match the regex <code>^[a-zA-Z0-9_-]{1,64}\$</code> .
<code>description</code>	A detailed plaintext description of what the tool does, when it should be used, and how it behaves.

<code>ma</code>	A JSON Schema object defining the expected parameters for the tool.
<code>input_examples</code>	(Optional) An array of example input objects to help Claude understand how to use the tool. See Providing tool use examples .

For the full set of optional properties available on any single tool definition, including `cache_control`, `strict`, `defer_loading`, and `allowed_callers`, see the [Tool reference](#). A client toolset entry accepts `cache_control` and `allowed_callers` on the entry and sets `defer_loading` per member; see [Client toolsets](#).

Example simple tool definition



Tool use system prompt

When you call the Claude API with the `tools` parameter, the API constructs a special system prompt from the tool definitions, tool configuration, and any user-specified system prompt. The constructed prompt is designed to instruct the model to use the specified tool(s) and provide the necessary context for the tool to operate properly:

```
In this environment you have access to a set of tools you can use to answer the user's question.
{{ FORMATTING INSTRUCTIONS }}
String and scalar parameters should be specified as is, while lists and objects should use JSON format. Note that spaces for string values are not stripped.
The output is not expected to be valid XML and is parsed with regular expressions.
Here are the functions available in JSONSchema format:
{{ TOOL DEFINITIONS IN JSON SCHEMA }}
{{ USER SYSTEM PROMPT }}
{{ TOOL CONFIGURATION }}
```

Best practices for tool definitions

To get the best performance out of Claude when using tools, follow these guidelines:

- When it should be used (and when it shouldn't)
 - What each parameter means and how it affects the tool's behavior
 - Any important caveats or limitations, such as what information the tool does not return if the tool name is unclear. The more context you can give Claude about your tools, the better it will be at deciding when and how to use them. Aim for at least 3–4 sentences for each tool description, more if the tool is complex.
- **Prioritize descriptions, but consider using `input_examples` for complex tools.** Clear descriptions are most important, but for tools with complex inputs, nested objects, or format-sensitive parameters, you can use the `input_examples` field to provide schema-validated examples. See [Providing tool use examples](#) for details.
 - **Consolidate related operations into fewer tools.** Rather than creating a separate tool for every action (`create_pr` , `review_pr` , `merge_pr`), group them into a single tool with an `action` parameter. Fewer, more capable tools reduce selection ambiguity and make your tool surface easier for Claude to navigate.
 - **Use meaningful namespacing in tool names.** When your tools span multiple services or resources, prefix names with the service (for example, `github_list_prs` , `slack_send_message`). This makes tool selection unambiguous as your library grows, and is especially important when using [tool search](#).
 - **Design tool responses to return only high-signal information.** Return semantic, stable identifiers (for example, slugs or UUIDs) rather than opaque internal references, and include only the fields Claude needs to reason about its next step. Bloated responses waste context and make it harder for Claude to extract what matters.

Example of a good tool description



Example poor tool description



The good description clearly explains what the tool does, when to use it, what data it returns, and what the `ticker` parameter means. The poor description is too brief and leaves Claude with many open questions about the tool's behavior and usage.



For deeper guidance on tool design (consolidation, naming, and response shaping), see [Writing tools for agents](#).

Providing tool use examples

You can provide concrete examples of valid tool inputs to help Claude understand how to use your tools more effectively. This is particularly useful for complex tools with nested objects, optional parameters, or format-sensitive inputs.

Add an optional `input_examples` field to your tool definition with an array of example input objects. Each example must be valid according to the tool's `input_schema` :

cURL CLI **Python** TypeScript C# Go Java PHP Ruby



```
client = anthropic.Anthropic()

response = client.messages.create(
    model="claude-opus-5",
    max_tokens=1024,
    tools=[
        {
            "name": "get_weather",
            "description": "Get the current weather in a given location",
            "input_schema": {
                "type": "object",
                "properties": {
                    "location": {
                        "type": "string",
                        "description": "The city and state, e.g. San Francisco,
CA",
                    },
                    "unit": {
                        "type": "string",
                        "enum": ["celsius", "fahrenheit"],
                        "description": "The unit of temperature",
                    },
                },
                "required": ["location"],
            },
            "input_examples": [
                {"location": "San Francisco, CA", "unit": "fahrenheit"},
                {"location": "Tokyo, Japan", "unit": "celsius"},
                {
                    "location": "New York, NY" # 'unit' is optional
                },
            ],
        },
    ],
    messages=[{"role": "user", "content": "What's the weather like in San
Francisco?"}],
)

print(response)
```

Requirements and limitations

- **Schema validation** - Each example must be valid according to the tool's `input_schema` . Invalid examples return a 400 error
- **Not supported for server-side tools or client toolsets** - Input examples work on user-defined and Anthropic-schema client tools other than the computer use and browser use toolsets, but not on server tools such as web search or code execution
- **Token cost** - Examples add to prompt tokens: ~20-50 tokens for simple examples, ~100-200 tokens for complex nested objects

Controlling Claude's output

Forcing tool use

In some cases, you may want Claude to use a specific tool to answer the user's question, even if Claude would otherwise answer directly without calling a tool. You can do this by specifying the tool in the `tool_choice` field of the request. The highlighted lines are the only difference from a standard tool use request:

cURL CLI **Python** TypeScript C# Go Java PHP Ruby



```
tools = [
    {
        "name": "get_weather",
        "description": "Get the current weather in a given location",
        "input_schema": {
            "type": "object",
            "properties": {
                "location": {
                    "type": "string",
                    "description": "The city and state, e.g. San Francisco,
CA",
                }
            },
            "required": ["location"],
        },
    },
]

response = client.messages.create(
    model="claude-opus-5",
    max_tokens=1024,
    tools=tools,
    tool_choice={"type": "tool", "name": "get_weather"},
    messages=[{"role": "user", "content": "What's the weather like in San
Francisco?"}],
)

print(response)
```

When working with the `tool_choice` parameter, there are four possible options:

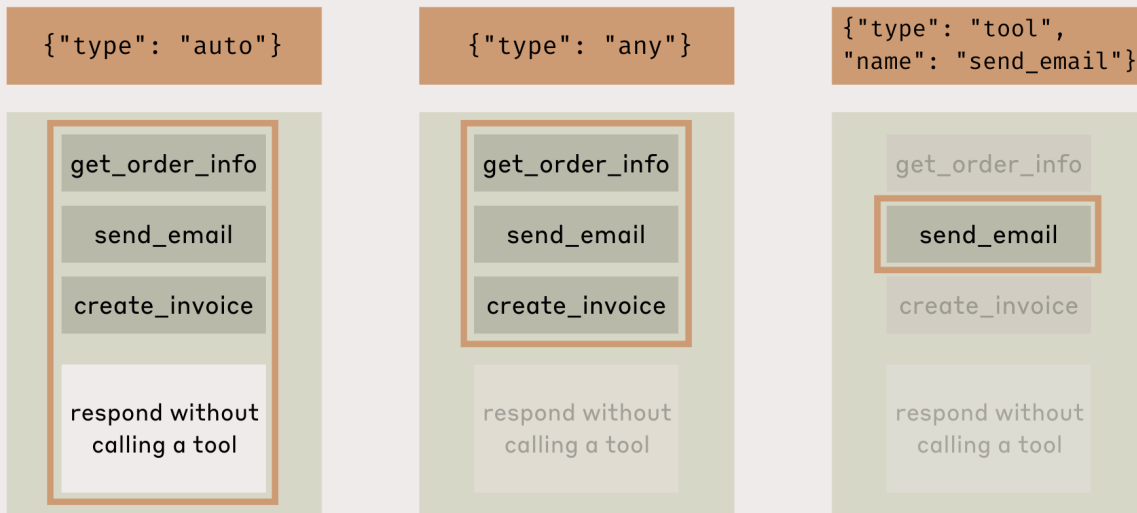
- `auto` allows Claude to decide whether to call any provided tools or not. This is the default value when `tools` are provided.
- `any` tells Claude that it must use one of the provided tools, but doesn't force a particular tool.
- `tool` forces Claude to always use a particular tool.
- `none` prevents Claude from using any tools. This is the default value when no `tools` are provided.



When using [prompt caching](#), changes to the `tool_choice` parameter will invalidate cached message blocks. Tool definitions and system prompts remain cached, but message content must be reprocessed.

This diagram illustrates how each option works:

Tool choice



Note that when you have `tool_choice` as `any` or `tool`, the API prefills the assistant message to force a tool to be used. This means that the models will not emit a natural language response or explanation before `tool_use` content blocks, even if explicitly asked to do so.

When using manual extended thinking (`thinking: {type: "enabled"}`) with tool use, `tool_choice: {"type": "any"}` and `tool_choice: {"type": "tool", "name": "..."}` are not supported and result in an error. Only `tool_choice: {"type": "auto"}` (the default) and `tool_choice: {"type": "none"}` are compatible with manual extended thinking. Adaptive thinking, including on models where thinking is on by default such as Claude Opus 5, supports forced tool use.

Claude Mythos Preview does not support forced tool use. Requests with `tool_choice: {"type": "any"}` or `tool_choice: {"type": "tool", "name": "..."}` return a 400 error on this model. Use `tool_choice: {"type": "auto"}` (the default) or `tool_choice: {"type": "none"}` and rely on prompting to influence tool selection.

Testing has shown that this should not reduce performance. If you would like the model to provide natural language context or explanations while still requesting that the model use a specific tool, you can use `{"type": "auto"}` for `tool_choice` (the default) and add explicit instructions in a user message. For example: What's the weather like in London? Use the `get_weather` tool in your response.

Combine `tool_use` & `type: "any"` with `strict_tool_use` to guarantee both that one of your tools will be called AND that the tool inputs strictly follow your schema. Set `strict: true` on your tool definitions to enable schema validation.

Model responses with tools

When using tools, Claude often comments on what it's doing or responds naturally to the user before calling tools.

For example, given the prompt "What's the weather like in San Francisco right now, and what time is it there?", Claude might respond with:

JSON



```
{
  "role": "assistant",
  "content": [
    {
      "type": "text",
      "text": "I'll help you check the current weather and time in San Francisco."
    },
    {
      "type": "tool_use",
      "id": "toolu_01A09q90qw90lq917835lq9",
      "name": "get_weather",
      "input": { "location": "San Francisco, CA" }
    }
  ]
}
```

This natural response style helps users understand what Claude is doing and creates a more conversational interaction. You can guide the style and content of these responses through your system prompts and by providing `<examples>` in your prompts.

It's important to note that Claude may use various phrasings and approaches when explaining its actions. Your code should treat these responses like any other assistant-generated text, and not rely on specific formatting conventions.

Next steps

Handle tool calls

Tool Runner (SDK)

Let the SDK handle the agentic loop automatically.

Tool reference

Directory of Anthropic-provided tools and optional properties.

Cette page vous a-t-elle été utile ?



Claude Platform Docs



Solutions

AI agents

Code modernization

Coding

Customer support

Financial services

Government

Higher education

K-12 teachers

Life sciences

Partners

Claude on AWS

Claude on Google Cloud

Learn

Blog

Courses

Use cases

Connectors

Customer stories

Engineering at Anthropic

Events

Powered by Claude

Service partners

Startups program

Company

Anthropic

Research

News

Responsible Scaling Policy

Security and compliance

Transparency

Help and security

Availability

Status

Support

Discord

Terms and policies

Privacy policy

Responsible disclosure policy

Terms of service: Commercial

Terms of service: Consumer

Usage policy